Time Series Forecasting with Decision Trees and XGBoost

Yahia Taraf

April 12, 2026

How Do We Use ML for Time Series?

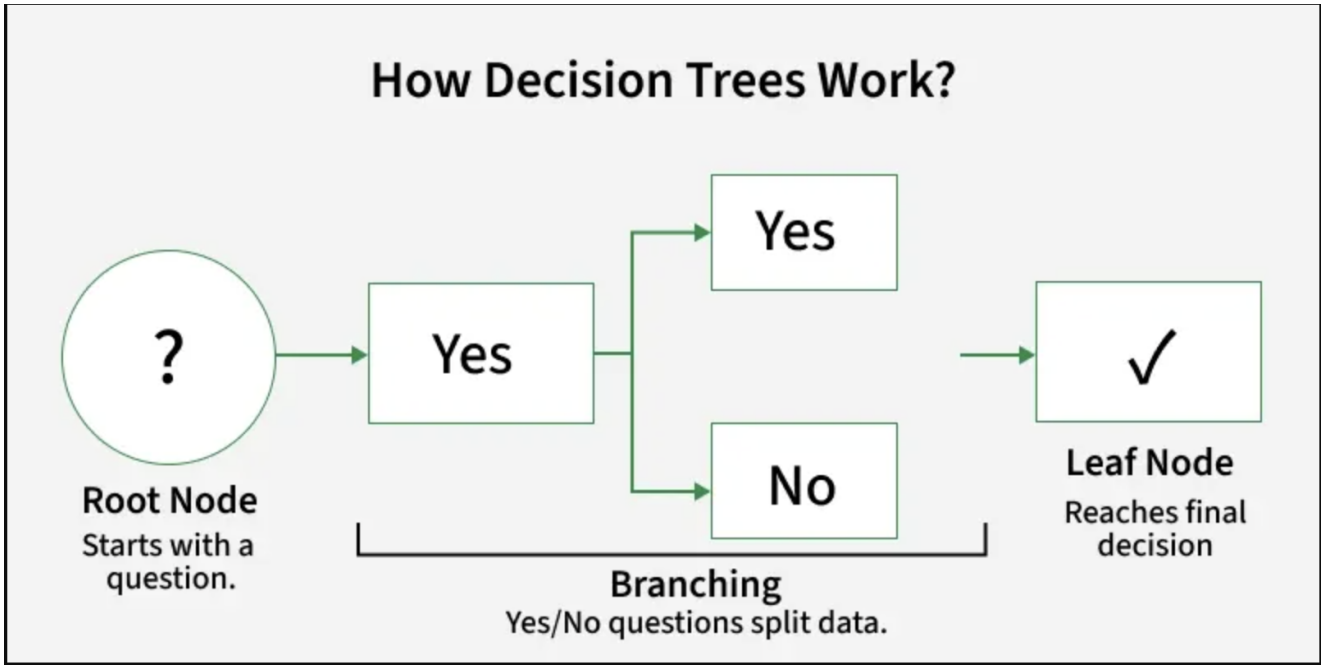
Traditional ARIMA models are considered Statistical Models and take a different approach than modern ML models.

To take a Machine Learning Approach we must

- Convert time series into supervised learning
- Example:
  - Input: past values (lags)
  - Output: next value
- This is called **feature engineering**

Machine Learning Approach: Decision Tree

- A model that splits data using yes/no questions
- Example:
  - Is yesterday's price ≥ 100?
  - Is volume increasing?
- Ends in a prediction (leaf node)



How Decision Trees Work and some Issues

- Split data based on feature thresholds
- Goal: reduce prediction error at each split
- Works well for:
  - Non-linear patterns
  - Mixed feature types

Problems with using a Single Trees

- Can overfit easily
- Sensitive to small data changes
- Not very stable for forecasting alone

What is Ensemble Learning & Gradient Boosting?

- **Ensemble Learning:** combining multiple models to improve performance
  - Instead of one model $\rightarrow$ many models working together
  - Reduces errors and improves stability
- Two main types:
  - **Bagging:** models trained independently (e.g., Random Forest)
  - **Boosting:** models trained sequentially (focus on mistakes)
- **Gradient Boosting:**
  - Each new model learns from previous errors (residuals)
  - Gradually improves predictions step-by-step

What is XGBoost?

- **XGBoost** stands for **Extreme Gradient Boosting**
- It is an advanced and highly optimized version of gradient boosting algorithms
- It builds many decision trees sequentially, where each tree improves on the errors of the previous ones
- It is designed to be:
  - Very fast (uses optimized computation and parallel processing)
  - Highly accurate (strong predictive performance)
  - Scalable (works well with large datasets)
- It also includes built-in techniques to reduce overfitting, such as regularization
- Because of its performance, it is widely used in:
  - Machine learning competitions (like Kaggle)
  - Real-world forecasting problems such as stock prediction and demand forecasting

How XGBoost Works (Intuition)

- Step 1: Make a simple prediction
- Step 2: Compute errors (residuals)
- Step 3: Train a tree on errors
- Step 4: Add corrections to prediction
- Repeat many times

Turning Time Series into XGBoost Input

- XGBoost cannot directly understand time order, so we convert time series into a supervised learning problem
- **Step 1: Create lag features (past values)**
  - $y_{t-1}, y_{t-2}, y_{t-3}$ represent previous time steps
  - These help the model learn short-term memory patterns
  - Example: yesterday's price helps predict today's price
- **Step 2: Add rolling statistics (trend information)**
  - Rolling mean $\rightarrow$ average of recent values (trend smoothing)
  - Rolling standard deviation $\rightarrow$ measures volatility or risk
  - Helps capture patterns like upward/downward trends
- **Step 3: Add time-based features (seasonality)**
  - Day of week (Mon, Tue, etc.)
  - Month or quarter of the year
  - Captures repeating seasonal patterns in data
- **Final idea:**
  - We transform time series into a table of features
  - XGBoost then learns patterns from this structured data

Example: Turning Time Series into a Feature Table

- We convert a time series into a supervised learning dataset
- Each row represents a **time snapshot**
- Input features = past values (lags)
- Target = the next value we want to predict

y(t-3)	y(t-2)	y(t-1)	y(t)
100	102	101	105
102	101	105	107
101	105	107	110

- Example interpretation:
  - Row 1: model uses (100, 102, 101) to predict 105
  - Row 2: model uses (102, 101, 105) to predict 107
  - Row 3: model uses (101, 105, 107) to predict 110
- Key idea: **XGBoost learns patterns from past → future mapping**
- This is how forecasting becomes a standard regression problem

Why XGBoost for Time Series? (Pros & Cons)

- XGBoost works very well for time series when we convert data into supervised learning using lag features
- **Why it works well:**
  - Captures complex non-linear patterns in data
  - Works very well with engineered features (lags, rolling averages, etc.)
  - Often faster than deep learning models on small to medium datasets
  - Provides strong baseline performance for forecasting problems
- **Advantages:**
  - High predictive accuracy
  - Robust to noise and outliers
  - Works well even with limited data
- **Limitations:**
  - Requires feature engineering (not end-to-end like LSTMs)
  - Does not naturally model sequences or time dependency